Productivity, Portability, Performance, and Reproducibility: Data-Centric Python

Alexandros Nikolaos Ziogas, Timo Schneider, Tal Ben-Nun, *Member, IEEE*, Alexandru Calotoiu, Tiziano De Matteis, Johannes de Fine Licht, Luca Lavarini, and Torsten Hoefler, *Fellow, IEEE*

***Abstract*—Python has become the *de facto* language for scientific computing. Programming in Python is highly productive, mainly due to its rich science-oriented software ecosystem built around the NumPy module. As a result, the demand for Python support in High-Performance Computing (HPC) has skyrocketed. However, the Python language itself does not necessarily offer high performance. This work presents a workflow that retains Python's high productivity while achieving portable performance across different architectures. The workflow's key features are HPC-oriented language extensions and a set of automatic optimizations powered by a data-centric intermediate representation. We show performance results and scaling across CPU, GPU, FPGA, and the Piz Daint supercomputer (up to 23,328 cores), with 2.47x and 3.75x speedups over previous-best solutions, first-ever Xilinx and Intel FPGA results of annotated Python, and up to 93.16% scaling efficiency on 512 nodes. Our benchmarks were reproduced in the Student Cluster Competition (SCC) during the Supercomputing Conference (SC) 2022. We present and discuss the student teams' results.**

***Index Terms*—Computer languages, Python, high-performance computing, dataflow computing, parallel programming, distributed computing.**

I. Introduction

PYTHON is *the* language to write scientific code [1]. The capability to write and maintain Python code with ease, coupled with a vast number of domain-specific frameworks and libraries such as SciPy, Matplotlib, scikit-learn [2], or pandas [3], leads to high productivity. It also promotes collaboration with reproducible scientific workflows shared using Jupyter notebooks [4]. Therefore, numerous scientific fields, ranging from machine learning [5], [6] to climate [7] and quantum transport [8] have already adopted Python as their language of choice for new developments.

As a result, the scientific community now pushes to also make Python *the* language for writing high-performance code. For most scientific applications, NumPy arrays [9] provide the core data structures, interfaces, and BLAS and LAPACK library interoperability. NumPy is optimized to provide efficient data structures and fast library implementations for many common operations. However, NumPy's performance benefits are tied to optimized method calls and vectorized array operations, both of which evaporate in larger scientific codes that do not adhere to these constraints. Therefore, there is a significant performance gap between the *numpythonic* code style and general code written by domain scientists.

In HPC, the three **P**s (**Productivity, Portability, Performance**) are driving recent developments in infrastructure and programming model research to ensure sustainability [10], [11]. In Python, this drive has resulted in many optimized domain-specific libraries and frameworks [5], [6], [7], [12], [13]. Simultaneously, the diversity of the hardware landscape motivated the creation of interface libraries, such as CuPy [14], which provides replacements to NumPy operations for NVIDIA and AMD GPUs, and MPI4PY [15], which offers direct MPI bindings. Lower-level interfaces, such as Cython [16], promise high performance at the cost of writing code that resembles C, and lazy evaluation of array operations [5], [17], [18] that enable high-performance runtime systems. Furthermore, a variety of JIT compilers [12], [19], [20] address the performance degradation resulting from the interpreter. Last but not least, runtime systems [18], distributed tasking (Dask) [21], and remote procedure calls [22] further scale Python to distributed systems. Despite the abundance of choices, Python still struggles: while each approach works towards one or more of the **P**s, none of them supports all at the same time.

We propose a way to bridge the gap between the three **P**s for Python programming using a data-centric paradigm. In particular, we empower Python users with an automatic optimization and specialization toolbox, which spans the entire Python/HPC ecosystem (Fig. 1) — from the code, through communication distribution, to hardware mapping. At the core of the toolbox, we use the Stateful Dataflow multiGraphs (SDFG) [23] data-centric intermediate representation, which enables these optimizations in the form of multi-level data movement transformations. With

Received 27 September 2024; accepted 11 October 2024. Date of current version 7 April 2025. This work was supported in part by the European Research Council (ERC) through the European Union's Horizon 2020 program under Grant 678880, Grant 801039, and Grant 955606 and in part by the Swiss National Science Foundation (SNSF) under Grant PZ00P2_185778, Grant 209358, and Grant 185778. The work of Alexandros Nikolaos Ziogas was supported by SNSF under Grant 209358. The work of Tal Ben-Nun was supported by SNSF under Grant 185778. *(Corresponding author: Alexandros Nikolaos Ziogas.)*

Alexandros Nikolaos Ziogas is with the Department of Information Technology and Electrical Engineering, ETH Zurich, 8092 Zurich, Switzerland (e-mail: Ziogasalziogas@iis.ee.ethz.ch).

Timo Schneider, Alexandru Calotoiu, and Torsten Hoefler are with the Department of Computer Science, ETH Zurich, 8092 Zurich, Switzerland.

Tal Ben-Nun is with the Center for Applied Scientific Computing, Lawrence Livermore National Laboratory, Livermore, CA 94550 USA.

Tiziano De Matteis is with the Department of Computer Science, Vrije Universiteit Amsterdam, 1081 HV Amsterdam, Netherlands (e-mail: t.de.matteis@vu.nl).

Johannes de Fine Licht is with the NextSilicon, 8005 Zurich, Switzerland.

Luca Lavarini is with the 1plusX, 8005 Zurich, Switzerland.

Digital Object Identifier 10.1109/TPDS.2025.3549310

1045-9219 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

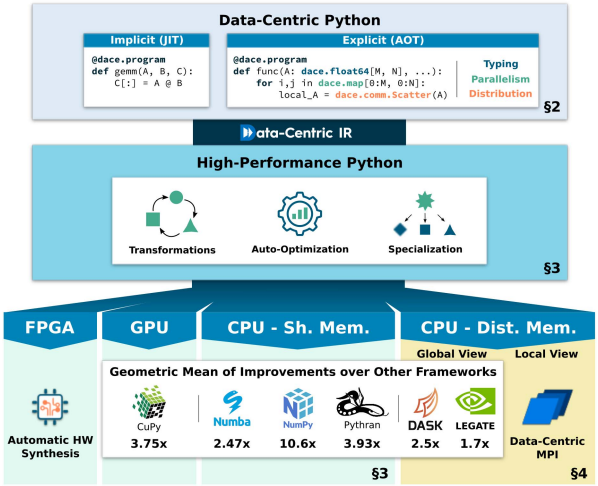


Fig. 1. Data-centric python overview.

a data-centric representation, as opposed to library bindings, all data dependencies and potential overlap are inferred statically from the code, and interpreter overhead is mitigated. Compared with implicit and lazy evaluation approaches, we also provide a set of extensions that give power users complete control over parallelism and partitioning schemes, using *pythonic* principles and syntax (e.g., returning "local view" objects from global data, but allowing users to operate on the "global view" as well).

We demonstrate a wide variety of benchmarks using the automatic toolbox **over annotated Python code**, both on individual nodes and the Piz Daint supercomputer. For the former, we show that it consistently outperforms other automatic approaches on multicore CPUs and GPUs, and *for the first time* show automatic Python HPC compilation results for both Xilinx and Intel FPGAs, which vastly differ in architecture and programming language. In distributed-memory environments, we show high scaling efficiency and absolute performance compared with distributed tasking. Thus, we realize all three **P**s within a single system.

Data-Centic Python was initially published at SC21 [24] and was selected for the SCC's Reproducibility Challenge at the SC22 conference. The student teams were tasked with reproducing our single- and multi-node results with NumPy and DaCe on their clusters or cloud infrastructure. We summarize their results and discuss the reproducibility of this work.

The paper makes the following contributions:

- (*Productivity*) Definition of high-performance Python, a methodology to translate it to a data-centric IR, and extensions to improve said conversion via explicit annotation.
- (*Portability*) A set of automatic optimizations for CPU, GPU, and FPGA, outperforming the best prior approaches by 2.47× on CPU and 3.75× on GPU on average (geometric mean [25]).
- (*Performance*) Automatic implicit MPI transformations and communication optimizations, as well as explicit distribution management, with the former scaling to 512 nodes with up to 93.16% efficiency.
- (*Reproducibility*) Discussion on the findings of the SCC22 student teams on the reproducibility of our experiments.

II. Data-Centric Python

The central tenet of our approach is that understanding and optimizing data movement is the key to portable, high-performance code. In a data-centric programming paradigm, three governing principles guide development and execution:

1) Data containers must be separate from computations.
2) Data movement must be explicit, both from data containers to computations and to other data containers.
3) Control flow dependencies must be minimized; they shall only define execution order if no implicit dataflow is given.

In the context of SDFGs, examples of data containers are arrays and scalar data, which have a NumPy-compatible data type, such as `int32` or `float64`.

Python is an imperative language and, therefore, not designed to express data movement. Its terseness makes the process of understanding dataflow difficult, even when comparing to other languages like C and FORTRAN, as the types of variables in Python code cannot be statically deduced.

Below, we define high-performance Python programs, discuss the decorators that we must add to Python code to make the dataflow analyzable, and then detail how we translate them into the SDFG data-centric intermediate representation.

A. High Performance Python

Our approach supports a large subset of the Python language that is important for HPC applications. The focus lies on NumPy arrays [9] and operations on such arrays. In addition to the low-overhead data structures NumPy offers, it is central to many frameworks focused on scientific computing, e.g., SciPy, pandas, and Matplotlib. As opposed to lazy evaluation approaches, high-performance Python must take control flow into account to auto-parallelize and avoid interpreter overhead. This tradeoff between performance and productivity is necessary because Python features such as co-routines are not statically analyzable and have to be parsed as "black boxes". To combat some of these Python quirks, we propose to augment the language with analyzable constructs useful for HPC.

B. Annotating Python

The Data-Centric (DaCe) Python frontend parses Python code and converts it to SDFGs on a per-function basis. The frontend will parse only the Python functions that have been annotated explicitly by the user with the `@dace.program` decorator. DaCe programs can then be called like any Python function and perform Just-in-Time (JIT) compilation.

a) Static symbolic typing: To enable Ahead-of-Time (AOT) compilation, which is of key importance for FPGAs and for reusing programs across different inputs, SDFGs should be statically typed. Therefore, the function argument data types are given as type annotations, providing the required information as shown below:

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

```
N = dace.symbol()
@dace.program
def jacobi_1d(TSTEPS: dace.int32,
              A: dace.float64[N],
              B: dace.float64[N]):

  for t in range(1, TSTEPS):
    B[1:-1] = 0.33333 * (A[:-2]+A[1:-1]+A[2:])
    A[1:-1] = 0.33333 * (B[:-2]+B[1:-1]+B[2:])
```

The Python method `jacobi_1 d` has three arguments; `TSTEPS` is a 32-bit integer scalar, while `A` and `B` are double-precision floating-point vectors of length `N`. The symbolic size `N`, defined with `dace.symbol`, indicates that the vector sizes can be dynamic (but equal). All subsets are then symbolically defined (e.g., the subset `B[1:-1]` becomes `B[1:N-1]`, and symbolic manipulation can then be performed in subsequent data-centric transformations.

b) Parametric parallelism: An important feature that has no direct expression in Python is a loop that can run in parallel. Our approach supports explicit parallelism declaration through map scopes, similarly to an N-dimensional `parallel for`. There are two ways to take advantage of this feature. DaCe provides the `dace.map` iterator, which can be used in Python code as a substitute to the Python built-in `range` iterator and generates a map scope when parsed:

```
for i, j in dace.map[0:M, 0:N]:
    A[i, j] = B[j, i]
```

Alternatively, the DaCe framework provides a `LoopToMap` transformation that detects for-loops in the IR, whose iterations can be executed safely in parallel (using symbolic affine expression analysis), and converts them to map scopes automatically.

C. From Python to DaCe

We turn to present the SDFG intermediate representation (IR) and a novel data-centric Python translation procedure in tandem. While previous work [23] converted a restricted, low-level Python definition of the SDFG IR, here we aim to cover the majority of the Python/NumPy language constructs via static analysis and fallback for unsupported features. We summarize the equivalence between Python constructs and SDFG counterparts in Table I, and present the generation of an SDFG from a Python program using the `gemm` kernel as an example:

```
@dace.program
def gemm(alpha, beta, C, A, B):
    C[:] = alpha * A @ B + beta * C
```

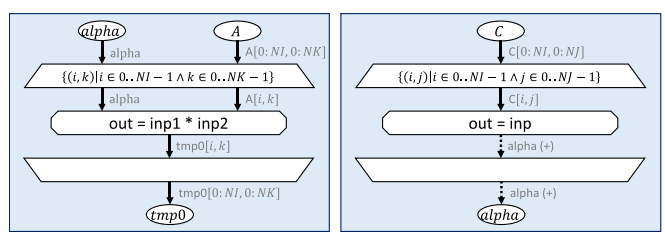
Our first pass traverses the Python AST to simplify the expressions into individual steps, similar to static single assignment [26], respecting the order of operations:

```
    tmp0 = alpha * A
    tmp1 = tmp0 @ B
    tmp2 = beta * C
    C = tmp1 + tmp2
```

The first step in the above code multiplies each element of `A` with `alpha`. SDFGs view data containers separately from the computations the data are part of, as per the first data-centric tenet. These containers are represented by oval *Access nodes*. In the first statement, these refer to `tmp0`, `alpha`, and `A` (see Fig. 2(a)).

TABLE I
MAPPING OF PYTHON SYNTAX AND CONSTRUCTS TO SDFG

Python	SDFG Equivalent
Declarations and Types	
Primitive data types	Scalar data container
NumPy array	Array data container
Assignments	
Assignments	Tasklet or map scope with incoming and outgoing memlets for read/written operands
Array subscript	Memlet (dataflow edge)
Statements	
Branching (`if`)	Branch conditions on state transition edges
Iteration (`for`)	Conditions, increments on state transition edges
Control-flow (`break`, `continue`, `return`)	Edge to control structure or function exit state
Functions	
Function calls (with source) and decorator for argument types	Nested SDFG for content, memlets reduce shape of inputs and outputs
External/Library calls	Tasklet with callback or Library Node



(a) Element-wise array operation `tmp0 = alpha * A`. (b) Augmented assignment with WCR.

Fig. 2. SDFG representations.

In SDFGs, connections to data containers are called *memlets*, and they describe the data movement — the edge direction indicates whether it is read or written, and its contents refer to the part of the data container that is accessed. Computations consume/produce memlets and can be divided into multiple types:

1) Stateless computations (*Tasklets*, shown as octagons), e.g., representing scalar assignments such as `a = 1`.
2) Calls to external libraries (*Library Nodes*, folded rectangles), that represent calls to functions that are not in the

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

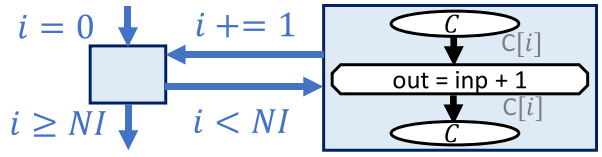


Fig. 3. SDFG representation of a Python for-loop. There are two states (left: guard, right: body) connected by control flow (state transition) edges that define the loop range.

list of functions decorated with `dace.program`. Matrix-matrix multiply is a common and important operation `tmp1 = tmp0 @ B` and is contained in a Library Node called *MatMul*.

3) Calls to other SDFGs (*Nested SDFGs*, rectangles), which represent calls to functions decorated with `dace.program`.
4) *Maps* are a particular type of Nested SDFGs matching the language augmentation previously discussed in Section II-B and express that the content can be processed in parallel.

Element-wise array operations automatically yield Map scopes. Similarly, assignments to arrays yield a Map scope, containing a Tasklet with the element-wise assignment. Augmented assignments such as `C += 1` are a special case where the output is also an input when no data races are detected.

Parallel maps can be augmented to express how *Write-Conflict Resolution* (WCR) should determine the value of data when multiple sources write to it concurrently. If data races are found, the outgoing edges are marked as dashed. E.g., the following program requires WCR (SDFG representation is shown in Fig. 2(b)):

```
for i, j in dace.map[0:NI, 0:NJ]:
    alpha += C[i, j]
```

By connecting the inputs and outputs of computations with the containers explicitly, the data-centric representation of a program can be created. To represent control dependencies, we encapsulate code driven by pure data dependencies in *States* (bright blue regions) in the SDFG. The states are connected with *State Transition Edges* (in blue) that can be annotated with control flow, representing loops, branch conditions, and state machines in general. The following Python program is represented by the SDFG in Fig. 3:

```
for i in range(NI):
    C[i] += 1
```

The above loop also is an excellent use case for the `LoopToMap` transformation (Section II-B) since its iterations are independent.

In the conversion to the SDFG IR, we also replace calls to library functions (e.g., `np.linalg.solve`) and object methods (`A.view()`) with custom subgraphs or Library Nodes, which users can extend for other libraries and object types via a decorated function. Following this initial conversion, the resulting SDFG contains a state per statement and a nested SDFG per function call.

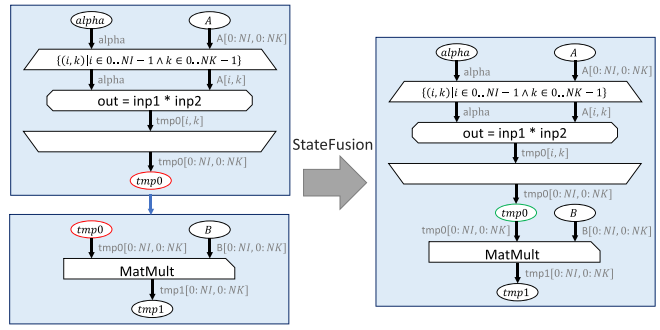


Fig. 4. State fusion of `tmp0=alpha*A` and `tmp1=tmp0@B`.

D. Dataflow Optimization

The direct translation to SDFGs creates a control-centric version of the code, which matches the Python semantics but does not contain dataflow beyond a single statement (similarly to compilers' `-O0`). To mitigate this, we run a pass of IR graph transformations that coarsens the dataflow, exposing a true data-centric view of the code (similar to `-O1`). The transformations include redundant copy removals, inlining Nested SDFGs, and others (14 in total), which only modify or remove elements in the graph, such that they cannot be applied indefinitely.

To understand this pass, we showcase one transformation — state fusion — which allows merging two states where the result does not produce data races. For example, the two states containing the assignments below can be merged:

```
tmp0 = alpha * A
tmp1 = tmp0 @ B
```

Internally, the transformation matches a subgraph pattern of two states connected together and compares source and sink Access nodes using symbolic set intersection. If no data dependency constraints are violated, Access nodes are either fused (if they point to the same memory, see Fig. 4) or set side by side, creating multiple connected components that can run in parallel.

All transformations in the DaCe framework follow the same infrastructure, either matching a subgraph pattern or allowing the user to choose an arbitrary subgraph [23]. While the dataflow coarsening pass happens automatically as part of our proposed toolbox, one can also apply transformations manually and separately, without changing the original Python source code (we color such "performance engineering codes" in cyan):

```
sdfg = gemm.to_sdfg()
sdfg.apply(StateFusion)
```

E. Python Restrictions

Some features available in Python are incompatible with our definition of high-performance Python and are discussed below. This does not exclude programs using the full feature

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

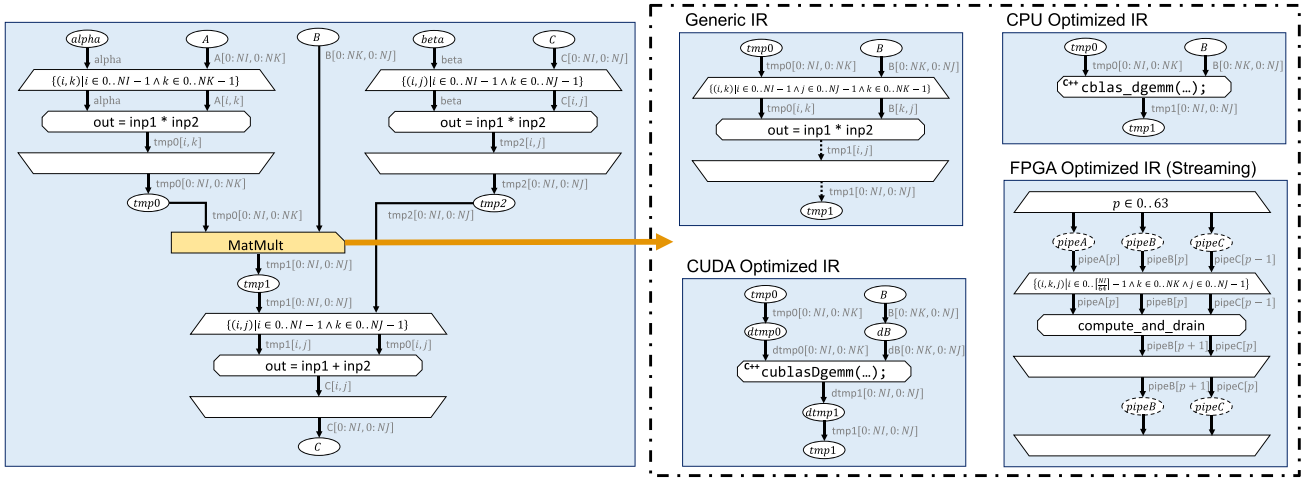


Fig. 5. Dataflow-coarsened GEMM SDFG and specializations for different architectures.

set of Python from analysis, but calls to functions containing unsupported features will not benefit from our optimization. The restricted features are:

1) Python containers (lists, sets, dictionaries, etc.) other than NumPy arrays as arguments, as they are represented by linked lists and difficult to analyze from a dataflow view. Note that this does not preclude internal containers (which can be analyzed) and list comprehensions.
2) Dynamic typing: fixed `structs` are allowed in SDFGs, so fields can be transformed and their class methods into functions decorated with the `dace.program` decorator. However, dynamic changes to classes or dynamic types are unsupported as their structure cannot be statically derived.
3) Control-dependent variable state (i.e., no scoping rules), e.g., the following valid Python:

```
x = ...
if x > 5:
  y = np.ndarray([5, 6], dtype=np.float32)
# use y (will raise exception if x <= 5)
```

4) Recursion. This is a limitation of the data-centric programming model [23], as recursion is a control-centric concept that is not portable across platforms.

After performing the full translation and coarsening, the resulting `gemm` kernel can be seen in Fig. 5 (left-hand side). This data-centric representation can now use the SDFG IR capabilities to further optimize and map the original Python code to different architectures and distributed systems.

III. Portability and Performance

The translated data-centric Python programs can now be optimized for performance on different hardware architectures. In this section, we propose a novel set of data-centric passes to auto-optimize and specialize SDFGs to run at state-of-the-art performance on CPUs, GPUs, and FPGAs, all from the same source IR. The programming portability is very high since our approach starts from the same Python codes parsed to the same SDFGs. Although the final optimized IRs may differ, the process can be automatic, and the user needs only to select the architecture to specialize for. In our evaluation, we only discuss results produced in an **automated** fashion.

A. Automatic Optimization Heuristics

As mentioned above, DaCe provides a user-extensible set of graph transformations. Yet, the framework does not perform them automatically [23], to endow performance engineers with fine-grained control and promote separation of concerns. Furthermore, DaCe includes tools and graphical interfaces to assist users with manual optimization without the explicit need for an expert. For productivity purposes, however, we believe that prototyping fast data-centric Python programs *should* be possible with minor code modifications.

By observing the common pitfalls in generated code from SDFGs versus what a performance engineer would write, we propose a set of transformation heuristics for SDFGs that yield reasonable performance in most cases (`-O3` compiler equivalent). This pass can be performed automatically (configurable) or using the following decorator:

```
@dace.program(auto_optimize=True, device=...)
```

where `device` can be `DeviceType.CPU,GPU,FPGA`.

Our auto-optimizer performs the following passes in order:

1) *Map scope cleanup:* Remove "degenerate" maps of size 1, repeatedly apply the *LoopToMap* transformation (Section II-B), and collapse nested maps together to form multidimensional maps. The latter also increases the parallelism of GPU kernels as a by-product.
2) *Greedy subgraph fusion:* Collect all the maps in each state, fusing together the largest contiguous subgraphs that share the same (or permuted) iteration space or the largest subset thereof (e.g., fusing the common three dimensions out of four). We use symbolic set checks on memlets to ensure that the data consumed is a subset of the data produced.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

3) *Tile WCR maps:* Tile (configurable size) parallel maps with write-conflicts that result in atomics, in order to drastically reduce such operations.
4) *Transient allocation mitigation:* Move constant-sized and small arrays to the stack, and make temporary data containers persistent (i.e., allocated upon SDFG initialization) if their size only depends on input parameters. This nearly eliminates dynamic memory allocation overhead.

Beyond the above general-purpose heuristics, we apply more transformations depending on the chosen device: For CPUs, we try to increase parallelism by introducing the OpenMP `collapse` clause. For GPU and FPGA, we perform the `GPU,FPGATransformSDFG` automatic transformations [23], which introduce copies to/from the accelerator and convert maps to accelerated procedures.

On the FPGA, we perform a few further transformations that diverge from the "traditional" fused codes in HPC: we create separate connected components (regions on the circuit) to stream off-chip (DRAM) memory in bursts to the program. Between computations, we try to modify the graph's structure to be composed of separate pipelined units that stream memory through FIFO queue Access nodes (we call this transformation `StreamingComposition`). This also enables further transformations to the graph to create systolic arrays during hardware specialization.

From this point, the only remaining step to lower the SDFG is to specialize the Library Nodes to their respective fastest implementations based on the target platform.

B. Library Specialization

Library Nodes, such as the `MatMul` operation in `gemm`, can be expanded to a wide variety of implementations. An *expansion* is defined similarly to a transformation, as a replacement subgraph to a single node, and can specialize its behavior.

We demonstrate specializations of matrix multiplication in Fig. 5. In particular, one can expand a Library Node into a C++ tasklet, calling an external function from MKL or CUBLAS (CPU, CUDA optimized IR in the figure); into an optimized subgraph (e.g., FPGA optimized IR, here using the Streamed expansion); or into a "native" SDFG subgraph (Generic IR).

In the automatic heuristics, we employ a priority list of implementations per platform, starting from the fast library calls, through optimized versions, and if all fail to expand (e.g., if the multiplication occurs within a kernel), we expand to the "native" SDFG. This yields an SDFG that can both be further optimized by performance engineers, and execute at reasonable performance out-of-the-box.

Users can employ the DaCe API to create their own libraries and Library Nodes [27]. This process is comparable to creating bindings.

C. Ahead-of-Time Compilation

DaCe provides extensive support for AOT compilation, due to the risk of overheads JIT compilation introduces in an HPC environment. Namely, DaCe provides the capability of AOT compilation if the decorated function's arguments are type-annotated as described in Section II-B. A decorated function or an SDFG can be directly compiled to a shared library through a Python script. Alternatively, the user can employ the *sdfgcc* command-line tool to compile SDFGs.

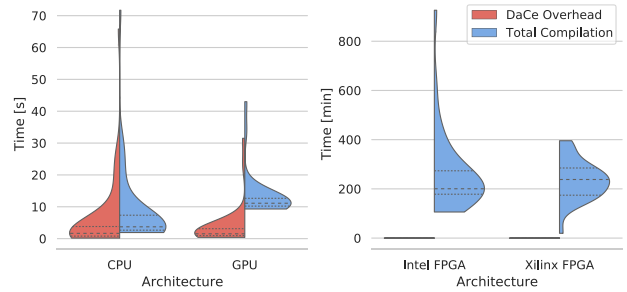


Fig. 6. Distributions of DaCe's total compilation times.

The compilation process utilizes specialized backends to generate optimized code for each supported architecture [23]. For example, C++ code is generated for CPUs, while the Nvidia GPU backend outputs C++ and CUDA programs. For FPGAs we utilize Vitis HLS and OpenCL for Xilinx and Intel architectures, respectively.

In Fig. 6 we present the distributions of DaCe's total compilation times on CPU, GPU, and FPGA for the set of benchmarks presented in Section III-D. This time includes parsing the Python code, auto-optimizing and compiling with GCC, NVCC, Intel OpenCL SDK, or the Xilinx Vitis compiler. 90% of the CPU and GPU codes compile in less than 15 s, while there is a single outlier above one minute. On FPGA architectures, synthesis, placement and routing typically takes hours, rendering DaCe's overhead negligible.

D. Evaluation

In the following, we show results for single node shared memory parallel programs created using data-centric Python for CPU, GPU and FPGA, and compare these with other frameworks: NumPy over the CPython interpreter, Numba, Pythran, and CuPy. We collect a set of existing Python codes from different scientific and HPC domains [7], [8], [28], [29], [30], [31], [32], [33], [34], [35], [36], [37], [38], [39], [40], [41], as well as a NumPy version of Polybench [42] ported from the C benchmark. In this adaptation, we strive to express the algorithms of the original benchmark in a way that is natural to a Python programmer. E.g., in `gemm`, `2mm`, and `3mm`, the matrix-matrix product is implemented with `@`, Python's dedicated operator for matrix multiplication [43]. All the data used are double-precision floating point numbers or 64-bit integers for CPU and GPU, while the FPGA tests use single-precision floating point numbers and 32-bit integers.

1) Experimental Setup: The CPU and GPU evaluations are performed on a machine running CentOS 8, with 1.5 TB of main memory, two Intel Xeon Gold 6130 CPUs (2x16 cores), and an NVIDIA V100 GPU (CUDA version 11.1) with 32 GB of RAM. We use CPython 3.8.5 as part of an Anaconda 3 environment.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

We test NumPy 1.19.2 with Intel MKL support, Numba 0.51.2 with Intel SVML support, the latest Pythran version from their GitHub repository [44] (commit ID 09349c5), and CuPy 8.3.0. For all frameworks that need a separate backend compiler, we use GCC 10.2.0, with all the performance flags suggested by the developers. To put high-performance Python into the perspective of low-level C implementations, we also compare the applications adapted from Polybench with the original Polybench/C [42] benchmark, compiled with GCC and the Intel C Compiler with automatic parallelization enabled (`icc -O3 -march=native -mtune=native -parallel`).

We evaluate FPGA performance on two different boards from either major FPGA vendor; a Bittware 520 N accelerator with an Intel Stratix 10 2800 GX FPGA and the Xilinx Alveo U250 accelerator board. Intel FPGA kernels are built with the Intel OpenCL SDK for FPGA and Quartus 20.3 targeting the `p520_max_sg280 h` shell, and Xilinx kernels are built with the Vitis 2020.2 compiler targeting the `xilinx_u250_xdma_201830_2` shell.

For the DaCe Python versions, we annotate types and symbolic shapes on the decorated functions to enable AOT compilation and work with FPGAs. We **do not** annotate loops as `dace.maps` and keep them in their original form, leaving parallelization for the automatic heuristics (Section III-A).

We compare the performance of the different frameworks and compilers using runtime as our primary metric of execution. Unless otherwise mentioned, we run each benchmark ten times and report the median runtime and 95% nonparametric confidence interval (CI) [45].

2) Benchmarking Results: CPU results are presented in Fig. 7. The right-most column contains NumPy's execution runtime for each of the benchmarks annotated on the y-axis. Each of the other columns contains the speedup (green tint and upward arrow) or slowdown (red tint and downward arrow) of execution compared to NumPy for each of the competing Python frameworks and Polybench C versions (compiled with ICC or GCC). Furthermore, we compute the 95% CI using bootstrapping [46] and annotate its size (as superscript in brackets) as a percentage of the median; values less than 1% are omitted (Fig. 7 uses vector graphics, and readers can zoom in with a PDF viewer if any values are not readable due to their size). The upper part of the chart aggregates the benchmarks using the geometric mean of the speedups over NumPy. Numba and Pythran have fallback modes for Python code failing to parse. In such cases, we measure the runtime of the fallback mode.

The figure shows an overall improvement in DaCe Python's performance, both over the Python compilers and the optimizing C compilers. Specifically, the subgraph fusion transformation capabilities surpass those of Numba. This is especially apparent in stencils, where the difference can be in orders of magnitude. In applications such as `crc16`, all compiled implementations successfully eliminate interpreter overhead. Shorter kernels benefit from the C versions due to runtime (and timing) overhead mitigation. It also appears that with control-flow heavy codes (e.g., `nussinov`), simple C code can be better optimized by GCC and ICC over generated code. It is worth noting that on some applications, NumPy is faster than the C versions: this is

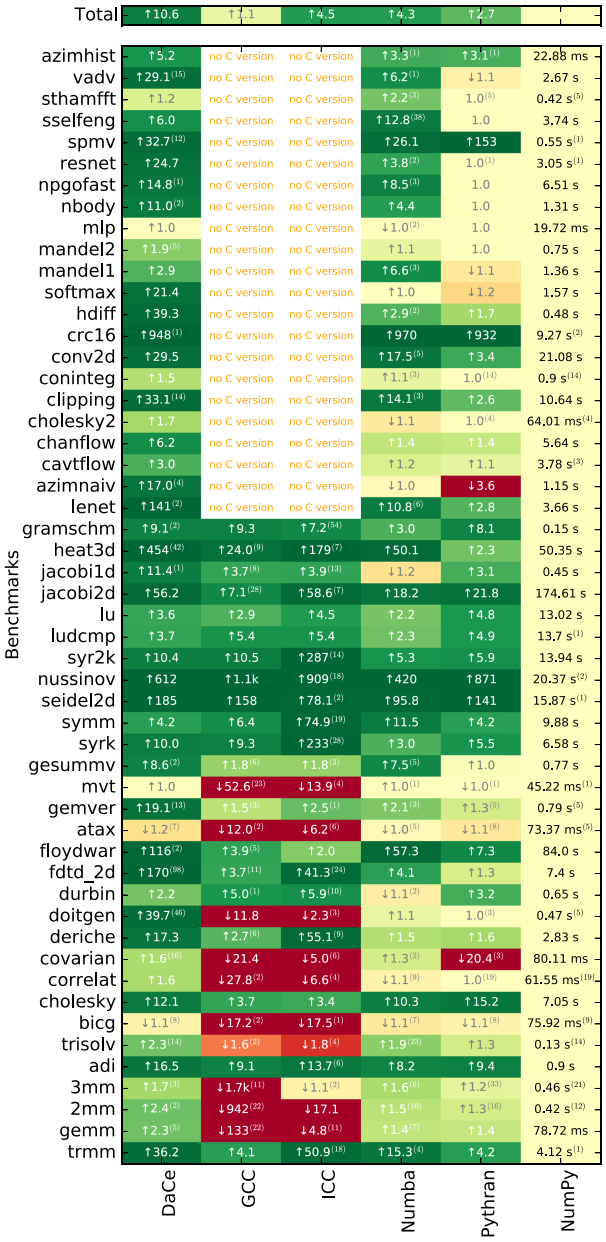


Fig. 7. CPU runtime and speedup over NumPy.

because of the performance benefits of vectorized NumPy code compared with explicit, sequential loops. An exceptional case is `3 mm`, which consists of three matrix multiplications. ICC pattern matches the matrix-matrix product and links to MKL, achieving similar performance to the Python frameworks. On the other hand, GCC does not compile the unoptimized C code to an executable that uses MKL, leading to lower performance.

Fig. 8 presents the runtime of applications that were successfully transformed to run on GPU. As with CPU, auto-optimizing DaCe consistently outperforms or is equivalent to CuPy, 3.75x (geomean) faster. The auto-optimization passes contribute to these results, mainly attributing to subgraph fusion and avoiding intermediate allocations on shorter applications. Due to redundant copy removal and view semantics being native to the SDFG,

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

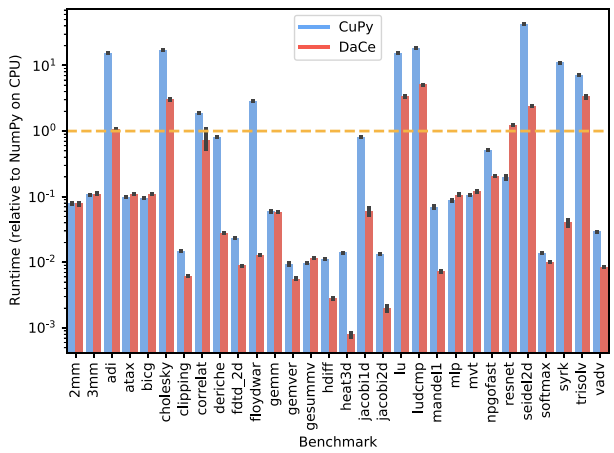


Fig. 8. CuPy and DaCe GPU runtime (lower is better).

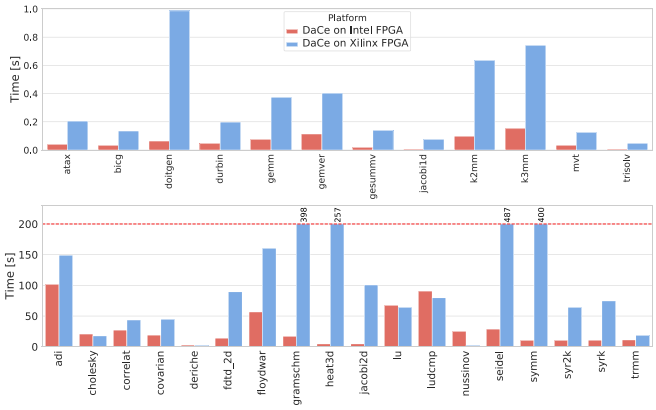


Fig. 9. FPGA runtime, `Large` instance, single precision.

we see a particular improvement on stencils, e.g., `heat3d`. Although CuPy-optimized code could potentially employ similar transformations [47], as far as we know, this cannot be performed out-of-the-box. The user must explicitly define element-wise or reduction based kernels, significantly changing the code. There is one instance where CuPy outperforms DaCe — `resnet`. This is due to a suboptimal vectorized representation of convolution in the Python source code, which translates to a loop of summations. In our generated code, this automatically results in many unnecessary atomic operations, even if tiled. The issue can be easily mitigated with further manual transformations (changing the maps' schedules) after the fact.

FPGA results can be seen in Fig. 9, where there is no comparison point as no other framework compiles high-performance Python directly. Although both platforms use different languages and features (e.g., accumulators), applications can be synthesized for both from the same annotated Python code. There is a noticeable difference in performance, especially on stencil-like applications, likely resulting from Intel FPGA's compiler toolchain superior stencil pattern detection. However, this can also be mitigated with subsequent manual transformations on the SDFG or augmenting the automatic heuristics decision-making process to transform stencils explicitly. Library Node expansions take device-specific features into account. For example, when there are accumulations (e.g., GEMV), we take advantage of hardened 32-bit floating-point accumulation on Intel FPGAs, which allows single-precision numbers to be directly summed into an output register. On Xilinx FPGAs, and, in general, for 64-bit floating-point accumulation, such native support does not exist. Therefore, we perform accumulation interleaving [48] across multiple registers to avoid loop-carried dependencies.

3) Discussion: While DaCe already outperforms existing libraries, this is not the end of the optimization process. Instead, the generated SDFGs can be *starting* points for optimization by performance engineers. The provided transformations and API can be used to eliminate sources of slowdown in the above applications or extended to use new, domain-specific transformations [27], [49]. Furthermore, the applications can also be adapted to distributed memory environments, where the productivity and performance benefits can be even greater.

IV. Scalable Distributed Python

The data-centric representation of Python programs can serve as a starting point for creating distributed versions. These distributed SDFGs abandon the global view of the data movement in favor of a local one. Like Message Passing, the flow of data in distributed memory is explicitly defined through Library Nodes. This approach allows for fine-grained control of the communication scheme and better mapping of SDFGs to code using optimized communication libraries.

In this Section, We show how to design transformations that specialize parallel map scopes to support distributed memory systems. We then show how to optimize such distributed data-centric programs. Finally, we show how developers can take control of the distribution entirely by expanding the original Python code with distributed communication while still allowing data-centric optimization to occur.

A. Transforming for Scale

Leveraging the data-centric representation, we can create transformations that convert specific shared-memory parallel kernels into distributed memory. The advantage of this approach is that once such a transformation is available, we can apply it to any subgraph in any SDFG that matches the same pattern. Furthermore, transformations can be compounded, building on each other to achieve complex results. We focus again on the `gemm` kernel for illustrating the transformations. As we shall show, the transformations extend beyond and automatically distribute other kernels as well.

a) Distributing global view element-wise operations: We distribute these by following a scatter-gather pattern, broadcasting (scalars) or scattering (arrays) input data containers from the root rank to the machine nodes, performing local computation, and gathering or reducing the outputs. By the nature of element-wise operations, careful selection of the array distribution parameters is not always necessary. The primary constraint is that each rank receives matching subsets of data, allowing it to perform local computation without further communication. Therefore, the most efficient distribution for contiguous arrays is to treat

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

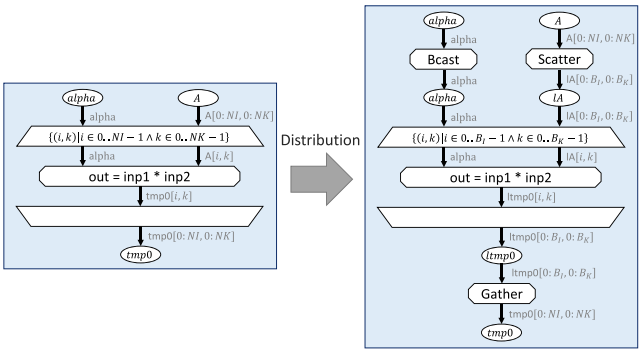


Fig. 10. Distribution of element-wise array operation.

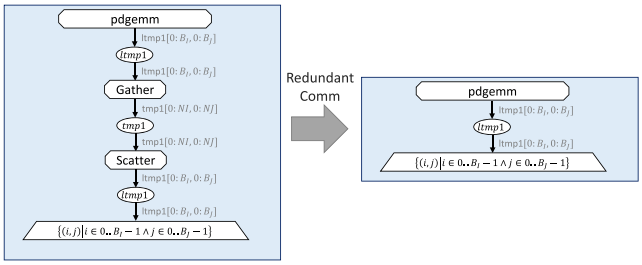


Fig. 11. Redundant communication elimination.

them as uni-dimensional and scatter them with `MPI_Scatter` (1-D block distribution). However, in cases where the result of an element-wise operation is consumed by, e.g., a matrix-matrix product or a stencil computation, it is beneficial to preserve the dimensionality of the arrays. For this reason, the transformation has optional parameters for the block sizes per dimension, leading to *block* or *block-cyclic* distributions. We offer implementations that use PBLAS [50] methods, such as `p?gemr2d` and `p?tran`, and MPI-derived data types, which have previously demonstrated performance benefits [51]. Applying the above transformation with *block* distributions on the operation `tmp0 = alpha * A` transforms the SDFG subgraph as shown in Fig. 10. We emphasize that these transformations are applied to an SDFG **without** changing the data-centric program code:

```
sdfg.apply(DistributeElementWiseArrayOp)
```

b) Distributing Library Nodes: We also create expansions for Library Nodes to distributed SDFG subgraphs. For example, the matrix-matrix and matrix-vector products expand to the aforementioned PBLAS library calls, along with the corresponding distribution of inputs and gathering outputs. Using PBLAS requires the definition of a process grid. The DaCe PBLAS library environment handles this automatically using BLACS [52]. The grid's parameters are free symbols that can be chosen by the user or take default values.

B. Optimizing Communication

Creating distributed versions of the operations separately from each other will perform correctly but poorly on real applications. Thus, we can use the data-centric aspect of the SDFG IR, which can track access sets through memlets to remove such communication bottlenecks automatically. Doing so separately from the distribution transformations allows users to write more pattern-matching distribution transformations without worrying about inter-operation communication, on the one hand; and on the other hand, allows the system to find such optimizations in any input code (e.g., if manually-written with communication redundancy).

One example of such a transformation is redundant gather-scatter removal. Consider the SDFG representation of `gemm`, shown in Fig. 5. We distribute all three element-wise array operations and we expand the `MatMult` node to a call to `pdgemm`. Due to the produced scatter and gather operations, the outputs `tmp1` of `pdgemm` and `tmp2` of `beta * C` will be connected to the last element-wise array operation `tmp1 + tmp2` as shown in the following **pseudo-code**:

```
pdgemm(..., ltmp1, ...)
tmp1 = gather(ltmp1)
ltmp1 = scatter(tmp1)
ltmp2 = beta * lC
tmp2 = gather(ltmp2)
ltmp2 = scatter(tmp2)
lC = ltmp1 + ltmp2
...
```

The above sequence of operations yields redundant communication on `tmp1` and `tmp2` and can therefore be omitted (the transformation for `tmp1` is shown in Fig. 11). By following the data movement and inspecting other Access nodes in the state, data dependencies of the global array can be inferred. Furthermore, since we know the `Scatter` and `Gather` node semantics, we can check whether the data distributions match. We note that users can use the DaCe API to define transformations to, e.g., optimize the re-distribution of data when the distributions do not match. If the distributions are 2D block-cyclic, such a transformation could, among other solutions, utilize PBLAS and a `p?gemr2d` Library Node to efficiently bypass the `Scatter` and `Gather` operations.

Combing the above transformations, the shared-memory `gemm` program from Section II-C can be converted to distributed-memory as follows, again without altering the code of the dace-centric Python program (type annotations omitted for brevity):

```
@dace.program
def gemm(alpha, beta, C, A, B):
    C[:] = alpha * A @ B + beta * C
```

```
dist_sdfg = gemm.to_sdfg()
dist_sdfg.apply(DistributeElementWiseArrayOp)
dist_sdfg.expand_library_nodes('PBLAS')
dist_sdfg.apply(RemoveRedundantComm)
```

C. Assuming Direct Control via Local Views

The implicit, global view approach works well for a plethora of Python programs that make heavy use of high-level array operations. However, as highly-tuned HPC applications often

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

```
@dace.program
def half_step(inpbuf: dace.float64[lNx+2, lNy+2],
              outbuf: dace.float64[lNx+2, lNy+2]):
  req = np.empty((8,), dtype=MPI_Request)
  dace.comm.Isend(inpbuf[1, 1:-1], nn, 0, req[0])
  # ...
  dace.comm.Irecv(inpbuf[1:-1, -1], ne, 2, req[7])
  dace.comm.Waitall(req)
  outbuf[1+noff:-1-soff, 1+woff:-1-eoff] = 0.2*(
      inpbuf[1+noff:-1-soff, 1+woff:-1-eoff] +
      # ...
      inpbuf[noff:-2-soff, 1+woff:-1-eoff])

@dace.program
def j2d_dist(TSTEPS: dace.int32,
             A: dace.float64[N, N],
             B: dace.float64[N, N]):
  lA = np.zeros((lNx+2, lNy+2), dtype=A.dtype)
  lB = np.zeros((lNx+2, lNy+2), dtype=B.dtype)
  lA[1:-1, 1:-1] = dace.comm.BlockScatter(A)
  lB[1:-1, 1:-1] = dace.comm.BlockScatter(B)
  for t in range(1, TSTEPS):
    half_step(lA, lB)
    half_step(lB, lA)
  A[:] = dace.comm.BlockGather(lA[1:-1, 1:-1])
  B[:] = dace.comm.BlockGather(lB[1:-1, 1:-1])
```

Fig. 12. Distributed `jacobi_2d` program.

use specific partitioning schemes, our data-centric toolbox also provides explicit control via Python annotations. As opposed to the existing tools that manage communication implicitly, the aim of the interface is to use *(num)pythonic* concepts to retain productivity while maximizing performance. E.g., the `jacobi_2d` stencil below would yield unnecessary `Scatter` and `Gather` collectives at every timestep:

```
@dace.program
def jacobi_2d(TSTEPS: dace.int32,
              A: dace.float64[N,N],
              B: dace.float64[N,N]):
  for t in range(1, TSTEPS):
    B[1:-1,1:-1] = 0.2 * (A[1:-1,1:-1] + A[1:-1,:-2]
    + A[1:-1,2:] + A[2:,1:-1] + A[:-2,1:-1])
    A[1:-1,1:-1] = 0.2 * (B[1:-1,1:-1] + B[1:-1,:-2]
    + B[1:-1,2:] + B[2:,1:-1] + B[:-2,1:-1])
```

For this reason, our data-centric approach allows the user to express arbitrary communication patterns by integrating explicit communication directly into Python. The above shared-memory data-centric Python program can be *modified* to be distributed. In the program shown in Fig. 12, we distribute the arrays `A` and `B` into 2D blocks at the beginning. The local views `lA, lB` are then computed and communicated via explicit halo exchange, using `Isend`, `Irecv`, and `Waitall` MPI calls in every time-step.

While this approach is similar to the *mpi4py* bindings, there are two distinct advantages to the data-centric approach with an explicit local view. First, the MPI calls are integrated into the program's dataflow with Library Nodes, enabling the above transformations and other automatic code generation features such as overlapping. Second, explicit *Isend* and *Irecv* calls communicate strided data using the MPI vector datatype, avoiding extraneous copies. The latter is also an example of using symbolic information on the graph to assert that high performance is attained — our MPI derived data type creation code, which is created once for each data type, relies on the symbol values not changing over the run. E.g., the initialization of the `inpbuf[1:-1, 1]` data type:

```
MPI_Datatype ntype;
MPI_Type_vector(lNx, 1, lNy+2, MPI_DOUBLE, &ntype);
MPI_Type_commit(&ntype);
```

would raise a performance warning in DaCe Python if the sizes may change at runtime. This avoids potential mistakes that even experienced performance engineers can make in large HPC codes.

D. Evaluation

To measure the performance of distributed data-centric programs, we conduct scaling experiments on the multi-core partition of the Piz Daint supercomputer. Each node has two 18-core Intel E5-2698v3 CPUs and 64 GB of memory. The nodes are connected through a Cray Aries network using a Dragonfly topology. We benchmark a subset of the Polybench kernels from Section III-D, which could be automatically transformed to use distributed memory (Section IV-A); `adi`, `bicg`, `doitgen`, `gemm`, `gemver`, `gesummv`, `k2mm`, `k3mm`, `mvt`, `jacobi_1d`, and `jacobi_2d`. We compare this work with Dask [21] v2.31 and Legate [18] (commit ID febd3bf [53]), two state-of-the-art distributed tasking Python frameworks, in weak scaling from 1 to 1,296 processes (648 nodes). Dask Array [54] scales a variety of NumPy workflows, including element-wise array operations, reductions, matrix-matrix products, and linear algebra solvers, among others. Legate is providing a drop-in replacement for the NumPy API to accelerate and distribute Python codes.

We select initial problem sizes that fit typical HPC workloads without having excessive runtime. The kernels' scaling factors and the initial problem sizes for each framework are presented in Table II. For kernels with computational complexity ranging from $O(n)$ to $O(n^2)$, we target a runtime in the order of 100ms. For kernels with higher complexity, we target a runtime in the order of 1 s. We note that with the problem sizes selected for benchmarking data-centric Python and Legate, Dask either runs out of memory or exhibits unstable performance. Thus, we halved the problem sizes for Dask but still encountered out-of-memory errors at larger node counts. Furthermore, several issues rendered testing Legate at scale difficult. With the assistance of the developers, many of those problems were solved; however, others remained. We could only run each benchmark up to a fraction of the total nodes available, either due to runtime errors or because a single execution did not finish within 10 minutes of allocated time. We annotate Fig. 13 with these errors.

We ignore the time needed for initializing and distributing data and only measure the main computation and communication time. We run all frameworks using default parameters where possible, i.e., "auto" as chunk size in Dask and block distributions (not block-cyclic, which would allow the user to fine-tune the block-sizes) on a 2D process grid for DaCe. Furthermore, we spawn one process per socket (2 processes per node) and 18 threads per process (equal to the number of physical cores in a socket). Legate is executed with parameters suggested by the

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

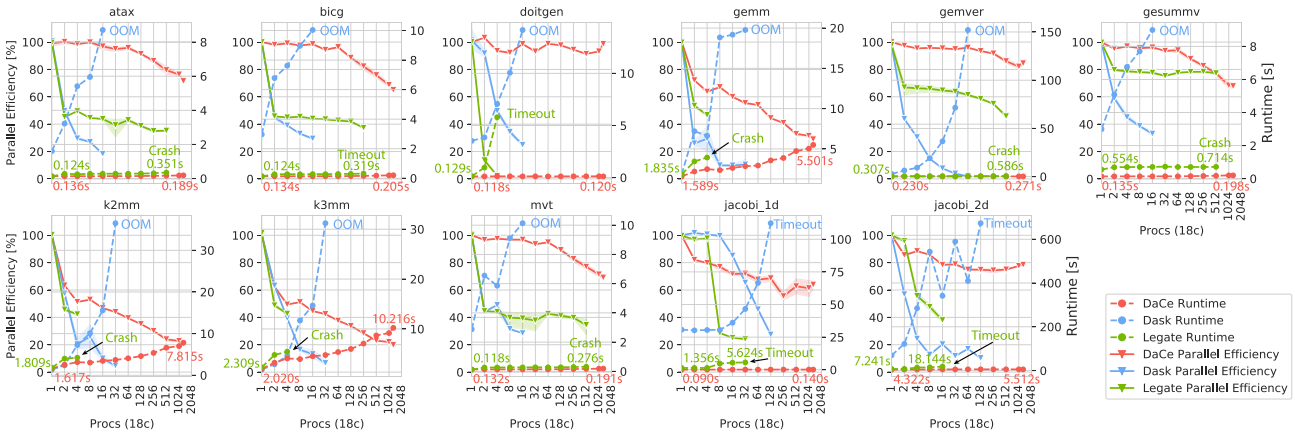


Fig. 13. Distributed runtime (dashed lines) and scaling efficiency (solid lines) on 23,328 cores of Piz Daint.

TABLE II
DISTRIBUTED BENCHMARKS, INITIAL PROBLEM SIZES FOR THE DIFFERENT FRAMEWORKS (F), AND SCALING FACTORS (S.F) AS A FUNCTION OF THE NUMBER OF PROCESSES S

Benchmark	F	Initial Problem Size	S.F
atax (M, N)	DaCe/Legate Dask	20000, 25000 10000, 12500	all $\sqrt{S}$
bicg (M, N)	DaCe/Legate Dask	25000, 20000 12500, 10000	all $\sqrt{S}$
doitgen (NR, NQ, NP)	DaCe/Legate Dask	128, 512, 512 128, 512, 512	$(S, -, -)$
gemm (NI, NJ, NK)	DaCe/Legate Dask	8000, 9200, 5200 4000, 4600, 2600	all $\sqrt[3]{S}$
gemver (N)	DaCe/Legate Dask	10000 5000	$\sqrt{S}$
gesummv (N)	DaCe/Legate Dask	22400 11400	$\sqrt{S}$
jacobi_1d (T, N)	DaCe/Legate Dask	1000, 24000 1000, 24000	$(-, S)$
jacobi_2d (T, N)	DaCe/Legate Dask	1000, 1300 1000, 1300	$(-, \sqrt{S})$
k2mm (NI, NJ, NK, NM)	DaCe/Legate Dask	6.4k, 7.2k, 4.4k, 4.8k 3.2k, 3.6k, 2.2k, 2.4k	all $\sqrt[3]{S}$
k3mm (NI, NJ, NK, NL, NM)	DaCe/Legate Dask	6.4k, 7.2k, 4.0k, 4.4k, 4.8k 3.2k, 3.6k, 2.0k, 2.2k, 2.4k	all $\sqrt[3]{S}$
mvt (N)	DaCe/Legate Dask	22000 11000	$\sqrt{S}$

developers: two NUMA domains per node, 28 GB of memory and one CPU with 16 threads per domain.

Fig. 13 presents the distributed runtime and scaling efficiency of DaCe, Dask, and Legate. DaCe exhibits four different efficiency patterns. In `doitgen`, the workload is distributed in an embarrassingly parallel manner, and no communication is needed. Therefore, the efficiency is close to perfect. The kernels `atax`, `bicg`, `gemver`, `gesummv`, and `mvt` compute matrix-vector products and scale very well until 64 processes, where the drop in efficiency becomes more pronounced, but remains above 60% for all data points. The matrix-matrix product kernels, `gemm`, `k2mm`, and `k3mm`, exhibit lower efficiency, which is consistent with the expected behavior of MKL-ScaLAPACK [55]. Finally, the stencil kernels' efficiency (`jacobi_1d` and `jacobi_2d`) falls between the last two categories of kernels. On the other hand, Dask and Legate exhibit a sharp drop in efficiency in almost all kernels immediately from the second process. An exception is the `jacobi_1d` kernel, where Dask has higher efficiency than DaCe up to 16 processes. However, this is possible due to Dask being much slower than DaCe (over 30x on the same problem size), allowing for much higher communication-computation overlap. On BLAS-heavy benchmarks, Legate matches the runtime of DaCe on a single CPU, whereas in others we observe slowdowns of 1.7–15×. In the benchmarks that scale to a large number of nodes (`atax`, `bicg`, `gemver`, `gesummv`, and `mvt`), Legate's efficiency, after the initial drop, remains constant.

DaCe uses MPI for communication and links to the optimized Cray implementation. Legate is built on top of the Legion runtime [56], which employs the GASNet library [57], a networking middleware implementing global-address space. Finally, Dask uses TCP for inter-worker communication. Although the different communication approaches cannot explain all performance discrepancies, there are the most significant factors in the overall picture.

V. PRODUCTIVITY

Python is already a very productive language, especially for domain scientists, due to its rich ecosystem described in Section I. Data-Centric Python is Python with extensions that themselves are valid Python syntax. For example, the `@dace.program` decorator follows the PEP 318 standard [58]. Therefore, Data-Centric Python essentially inherits the Python language's programming productivity. Since performance is the most important metric in HPC, scientific applications must eventually be lowered to a representation that is amenable to low-level optimizations for the underlying architectures. Traditionally, this translates to writing these applications in C, C++, and Fortran, among other device-specific languages. Therefore, an HPC project must force the domain scientists to sacrifice productivity and work directly on the lower-level

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

languages or maintain two different code bases. DaCe, and other frameworks that accelerate Python, increase HPC productivity by bridging the code that domain scientists want to write with the code that achieves high performance.

VI. Related Work

Approaches similar to our own targeting Python code have already been introduced and compared with in Sections I, III, and IV. In this section, we further discuss relevant frameworks, libraries, and approaches towards the three **P**s.

a) Productivity: The complexity of optimizing applications, combined with the repetitive nature of performance engineering for specific domains, has given rise to a wide variety of Domain-Specific Languages (DSLs) [59], [60], [61], [62] and embedded DSLs, particularly in Python [7], [13]. In the latter category, a notable example is deep learning frameworks, which use Python's various capabilities to construct readable code that performs well. PyTorch [5] uses object-oriented programming to construct deep neural networks as modules, relying on reflection to detect parameters and nonblocking calls for asynchronous execution to avoid interpreter overhead. TensorFlow [6] used Python's weak typing system to construct graphs from Python functions but has recently transitioned to "eager" execution to improve productivity, making codes more readable and simplifying debugging.

b) Portability: In the past three decades, compilers have undergone a transition from all-pairs solutions (between source languages and hardware platforms) to funneling through Intermediate Representations (IR), on which they can perform language- and platform-agnostic optimization passes. Although DaCe is currently developed in Python, many other research compilers are based on the LLVM [63] infrastructure and IR. There is an ongoing movement in the compiler community towards Multi-Level IRs [64], in which a multitude of IR *dialects* can retain domain- and platform-specific information, in turn enabling domain-specific optimizations [65]. MLIR performs optimization passes on each dialect to compile programs, followed by lowering passes to subsequent dialects, down to hardware mapping. This feature could be utilized to implement DaCe Library Nodes. The data-centric transformation API is also shared by languages such as Halide [66], which enables users to invoke schedule optimizations separately from program definition.

c) Performance portability: Aimed at keeping a consistent ratio of performance to peak performance across hardware [67], it is the core premise of several standards [68], [69], [70], [71], [72]. In directive-based frameworks [68], [69], *pragma* statements are added to C/C++ and FORTRAN programs to introduce parallelism, similarly to our proposed annotations. In kernel-based frameworks [70], [71], [72], *kernels* are constructed as functions with a limited interface and offloaded to target devices, such as CPUs, GPUs, or FPGAs. As each platform requires its own set of directives, kernel parameters, or sometimes kernel implementations, programs often contain multiple codes for each target. To resolve such issues, HPC languages such as Chapel [73] and HPF [74] define high-level implicit abstractions used as parallel primitives. Also popular in the HPC world are performance-portable libraries, embedded within C++, notably Kokkos [75], RAJA [76], and Legion [56] (which powers Legate [18]). These allow integrating heterogeneous and distributed systems through task-based abstractions, data dependency analysis (e.g., Legion's logical regions), and common parallel patterns. Such patterns can also be found in NumPy, and the SDFG can be seen as a generalization of these graphs with symbolic data dependencies.

VII. Reproducibility

HPC technologies are progressing at an accelerated pace. Although this rate of technological advancement provides the scientific community with much-needed computational power, HPC-related undergraduate curricula struggle to keep up. SC has taken steps to address the issue, introducing the Students@SC program. The program aims to educate students in the HPC field and assist their transition to HPC professionals. As part of this program, the Student Cluster Competition (SCC) has been held annually since 2007, demonstrating how everyone can use HPC. In this 48-hour challenge, competing undergraduate student teams learn how to use scientific applications and, most importantly, optimize them to extract as much performance as possible from small clusters of their own design.

Another major SC initiative aims to improve the reproducibility of HPC-related publications. To that end, SCC includes a *reproducibility challenge*, where the students attempt to reproduce results from an accepted paper from SC's Technical Program. For SCC22, we were chosen to design the challenge based on our SC21 publication on Data-Centric Python. We asked the students to replicate the results of our benchmarks using DaCe and the NumPy-based baselines for shared and distributed memory. The students could use the software versions noted in Sections II-I-D and IV-D, newer ones, or mix them. We also provided the competition with a secret task, where the students needed to port three new Python microbenchmarks to DaCe and report on their findings. Below, we present the results of the challenge as they relate to productivity, portability, and performance.

A. Productivity

According to SCC's rules, the challenge's tasks were sent to the teams in advance so that they could prepare before the competition. An exception to that was the secret task, which was revealed at the beginning of the competition and involved three microbenchmarks from a set of NumPy benchmarks [77] written by the authors of Pythran: `lstsqr`, `specialconvolve`, and `wdist`. The students had to add missing type annotations to enable AOT compilation and amend the code to work around unsupported NumPy methods and Python syntax. The secret task was designed to be accessible to anyone who studied the original publication. Indeed, almost all competing teams completed the task with no issues. Although the programs were relatively small, the teams' success supports our claims on Data-Centric Python's productivity.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

TABLE III
SCC22 PARTICIPANT TEAMS, THEIR CPU MACHINES, THEIR ACHIEVED SINGLE-NODE CPU SPEEDUP (GEOMETRIC MEAN) OVER NUMPY WITH DACE, THE NUMBER OF BENCHMARKS EXHIBITING HIGH VARIANCE (95% CI IS 10% OR MORE OF THE MEDIAN RUNTIME), AND THEIR RUNTIMES IN SECONDS FOR THE `mandel1` AND `resnet` BENCHMARKS

Team	CPU Machine	Framework	Overall Speedup	Benchmarks with High Variance	Runtime [s] `mandel1`	Runtime [s] `resnet`
This work	Intel Xeon Gold 6130 32 Cores (2x16)	NumPy			1.36	3.05
		DaCe	10.6×	9/52	0.47	**0.12**
National Tsing Hua University	Intel Xeon Platinum 8168 44 Cores (2x22)	NumPy			2.57	3.83
		DaCe	10.7×	19/52	0.01	8.04
Zhejang University	Intel Xeon Platinum 8168 44 Cores (2x22)	NumPy			1.12	2.96
		DaCe	9.4×	0/52	0.02	0.74
University of California San Diego	AMD EPYC 7773X 128 Cores (2x64)	NumPy			4.69	4.84
		DaCe	7.6×	17/52	0.01	11.62
Sun Yat-sen University	AMD EPYC 7V73X 120 Cores (2x60)	NumPy			1.26	1.8
		DaCe	6.3×	16/52	**<0.01**	8.10
ETH Zurich	AMD EPYC 7773X 128 Cores (2x64)	NumPy			1.93	1.68
		DaCe	3.4×	33/52	0.03	15.46

The best DaCe results are highlighted in bold.

B. Portability

The SCC22 teams succeeded in running DaCe and the benchmarks on various hardware based on AMD, Intel, and NVIDIA architectures. As anticipated, the single-node CPU tests ran without issue for all teams since DaCe generates C++14 code with OpenMP, which is supported in all major CPU-based HPC platforms. GPU tests were executed successfully on NVIDIA V100 and A100 GPUs. However, teams were unsuccessful in running the benchmarks on AMD GPU devices. Our provided benchmark infrastructure (NPBench [78]) isolates the program execution on the GPU device from the data transfers from/to the host. It uses CuPy to copy the data to the device, extract the GPU global memory pointers, and pass them to the executable programs. When the competition was held, there was an issue with CuPy and AMD's GPU toolchain (HIP/ROCm) that did not allow the second step above to succeed. The issue has since been resolved. Distributed tests were also run without problems, as the students could utilize DaCe's infrastructure to link their programs against the MPI and ScaLAPACK libraries installed on their systems.

C. Performance

Although all teams observed accelerated program execution using DaCe, the actual performance varied, especially in the single-node CPU experiments. In contrast, the teams' GPU execution matched ours much more closely since they used the same (NVIDIA V100) or very similar (NVIDIA A100) hardware. In Table III, we present the CPU hardware the teams used for those benchmarks and their achieved DaCe-speedup over NumPy. Two teams used Intel-based (Skylake architecture) machines with 44 cores, while the rest used AMD CPUs (Milan architecture) with 120/128 cores. The former received overall results that were much closer to ours. Furthermore, the latter team recorded a higher variance. While in our experiments, nine benchmarks have a 95% CI that is 10% or more of the median runtime, the teams using AMD CPUs observed up to 33 benchmarks with such values. To shed further light on the performance discrepancies, we also look closer at the performance of two benchmarks, `mandel1` and `resnet`, which exhibit large deviations between our and the teams' tests.

The two architectures used, Skylake and Milan, have very different core topologies. The Intel CPUs are monolithic dies and have uniform core-to-core intra-socket latencies. In contrast, the AMD CPUs consist of *chiplets* of 8 cores each and exhibit the lowest latency in intra-chiplet communication. Therefore, thread placement is much more critical on the AMD CPUs than on the Intel-based machines tested. For codes utilizing OpenMP for multicore execution, such as those generated by DaCe, there are environment variables related to the OpenMP runtime that explicitly control thread binding and placement. However, DaCe does not set those variables automatically, potentially performing suboptimally in architectures sensitive to those options, as seen in Table III. Future work could provide improvements by querying hardware information, for example, using the Portable Hardware Locality (`hwloc`) [79] software package and providing tuning guidance to the users.

All teams discussed used the at-the-time current DaCe release (0.14.1), which included changes in the automatic optimization heuristics presented in Section III-A. One of the most significant changes was the automatic specialization of the Library Node representing reductions. In our original experiments and for CPU execution, DaCe replaced the node with a map scope with WCR, subsequently tiled to reduce atomic operations. In the version tested by the teams, our auto-optimization routine specialized the node with an implementation using the OpenMP reduction clause without tiling. This implementation proved suboptimal when performing partial reductions on multi-dimensional arrays, causing false sharing and severely reducing performance in many benchmarks, such as `lenet` and `resnet`. The teams' results using AMD CPUs and, therefore, up to four times as many cores compared to our setup, were particularly affected. We

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

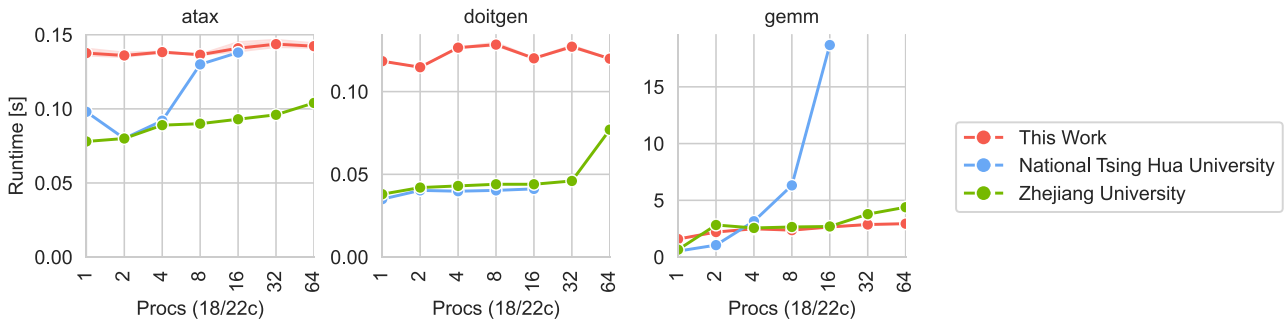


Fig. 14. Distributed runtime for `atax`, `doitgen`, and `gemm` for this work and two SCC22 teams up to 64 processes.

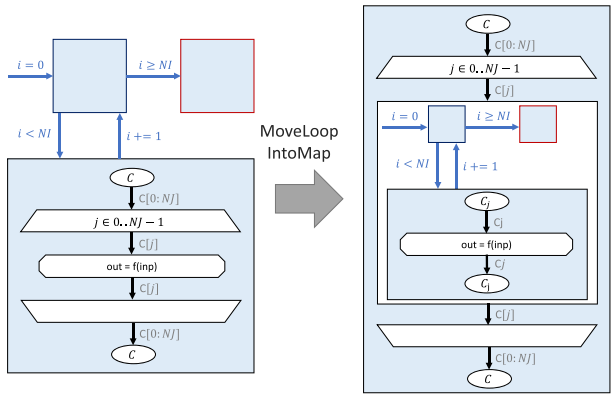


Fig. 15. MoveLoopIntoMap: Interchanging loop and map scopes.

note that in later DaCe releases, the above change was reverted, resolving the performance degradation.

On the other hand, there were also significant performance improvements, as seen in benchmarks such as `mandell`, `symm`, and `syr2k`. For example, `mandell` benefited from a new transformation integrated into DaCe's automatic optimization heuristics, *MoveLoopIntoMap*, shown in Fig. 15. This transformation matches a for-loop with a nested map scope in its body. If no data dependencies are violated, the map and the loop iteration spaces are interchanged, parallelizing the outer scope. The optimization reduces parallelization overheads and may significantly increase temporal locality and performance.

The teams largely matched our parallel efficiency trends in distributed-memory benchmarks: memory-bound benchmarks such as `atax` exhibited higher parallel efficiency than compute-bound codes, e.g., `gemm`. Fig. 14 shows the runtimes for our experiments and the two Intel-based teams that matched our hardware more closely, National Tsing Hua University and Zhejiang Univesity, up to 64 CPUs. We note that the teams' compute runtimes were generally lower due to their faster CPUs but had lower communication performance due to the use of Ethernet for communication. An exception is the results from the Sun Yat-set University team, who, due to a lack of working machines, assigned 32 processes to just two nodes, causing increased contention for memory bandwidth. Therefore, their results exhibited much lower parallel efficiency for memory-bound codes, which was expected under the circumstances.

VIII. Conclusion

While distributed SDFGs forego the global view of data movement to facilitate the design of custom communication schemes, future work could explore the trade-offs between a more *pythonic* approach to communication and extracting the best performance. Moreover, improvements to existing transformations, e.g., Vectorization, and implementation of new ones could increase the out-of-the-box performance, reducing the need for manual optimization.

We present Data-Centric Python — a high-performance subset of Python with annotations that produces supercomputer-grade HPC codes. Based on the SDFG intermediate representation, we show how a pipeline of static code analysis and dataflow transformations can take input NumPy code, leverage its vectorized nature, and map it efficiently to CPUs, GPUs, and FPGAs, outperforming current state-of-the-art approaches on each platform by at least 2.4x.

The resulting data-centric programs effectively eliminate the performance pitfalls of Python, including interpreter overheads and lack of dataflow semantics for library calls, the latter being crucial for running at scale. Many even outperform baselines written in C code. A key feature of the data-centric toolbox is giving users explicit control when necessary rather than making assumptions at the framework level. We evaluate Data-Centric Python in a distributed environment and show that the parallel efficiency remains above 90%, even on hundreds of nodes. All student teams at the Student Cluster Competition at SC22 largely reproduced our experiments, further strengthening our claims of portability and productivity. These promising results indicate that productive coding with Python can scale *and* map to heterogeneous compute architectures, setting the once-scripting language at the same level as FORTRAN, C, and other HPC giants.

Acknowledgment

The authors would like to thank Mark Klein and the Swiss National Supercomputing Centre (CSCS), Paderborn University

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

(DaceML-FPGA project), and AMD/Xilinx (HACC program) for support and access to computational resources.

References

[1] GitHub, "The 2020 state of the octoverse," 2020. [Online]. Available: https://octoverse.github.com/

[2] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[3] W. McKinney, "Data structures for statistical computing in python," in *Proc. 9th Python Sci. Conf.*, 2010, pp. 56–61.

[4] T. Kluyver et al., "Jupyter notebooks - A publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, Amsterdam, Netherlands: IOS Press, 2016, pp. 87–90. [Online]. Available: https://eprints.soton.ac.uk/403913/

[5] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

[6] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: http://tensorflow.org/

[7] Swiss National Supercomputing Centre (CSCS), "Gt4py," 2014. [Online]. Available: https://github.com/GridTools/gt4py

[8] A. N. Ziogas, T. Ben-Nun, G. I. Fernández, T. Schneider, M. Luisier, and T. Hoefler, "A data-centric approach to extreme-scale ab initio dissipative quantum transport simulations," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2019, Art. no. 1.

[9] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.

[10] S. Nanz and C. A. Furia, "A comparative study of programming languages in rosetta code," in *Proc. 37th Int. Conf. Softw. Eng.*, 2015, pp. 778–788.

[11] T. Ben-Nun, T. Gamblin, D. S. Hollman, H. Krishnan, and C. J. Newburn, "Workflows are the new applications: Challenges in performance, portability, and productivity," in *Proc. 2020 IEEE/ACM Int. Workshop Perform. Portab. Productiv. HPC*, 2020, pp. 57–69.

[12] J. Bradbury et al., "JAX: Composable transformations of Python NumPy programs," 2018. [Online]. Available: http://github.com/google/jax

[13] M. Lange et al., "Devito: Towards a generic finite difference DSL using symbolic python," in *Proc. 6th Workshop Python High- Perform. Sci. Comput.*, 2016, pp. 67–75.

[14] P. Networks, "Cupy," 2021. [Online]. Available: https://cupy.dev/

[15] L. D. Dalcin, R. R. Paz, P. A. Kler, and A. Cosimo, "Parallel distributed computing using python," *Adv. Water Resources*, vol. 34, no. 9, pp. 1124–1139, 2011. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0309170811000777

[16] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith, "Cython: The best of both worlds," *Comput. Sci. Eng.*, vol. 13, no. 2, pp. 31–39, 2011.

[17] M. R. Kristensen, S. A. Lund, T. Blum, K. Skovhede, and B. Vinter, "Bohrium: Unmodified numpy code on CPU, GPU, and cluster," in *Proc. Workshop Python High Perform. Sci. Comput.*, 2013, pp. 1–8.

[18] M. Bauer and M. Garland, "Legate numpy: Accelerated and distributed array computing," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, New York, NY, USA, 2019, Art. no. 23, doi: 10.1145/3295500.3356175.

[19] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based python JIT compiler," in *Proc. 2nd Workshop LLVM Compiler Infrastructure HPC*, New York, NY, USA, 2015, Art. no. 7, doi: 10.1145/2833157.2833162.

[20] S. Guelton, P. Brunet, M. Amini, A. Merlini, X. Corbillon, and A. Raynaud, "Pythran: Enabling static optimization of scientific python programs," *Comput. Sci. Discov.*, vol. 8, no. 1, 2015, Art. no. 014001.

[21] Dask Development Team, "Dask: Library for dynamic task scheduling," 2016. [Online]. Available: https://dask.org

[22] P. Moritz et al., "Ray: A distributed framework for emerging AI applications," in *Proc. 13th USENIX Conf. Operating Syst. Des. Implementation*, USA: USENIX Association, 2018, pp. 561–577.

[23] T. Ben-Nun, J. de Fine Licht, A. N. Ziogas, T. Schneider, and T. Hoefler, "Stateful dataflow multigraphs: A data-centric model for performance portability on heterogeneous architectures," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2019, Art. no. 81.

[24] A. N. Ziogas et al., "Productivity, portability, performance: Data-centric python," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, New York, NY, USA, 2021, Art. no. 95, doi: 10.1145/3458817.3476176.

[25] Standard Performance Evaluation Corporation, "CPU 2017 metrics," Aug. 2020. [Online]. Available: https://www.spec.org/cpu2017/Docs/overview.html#metrics

[26] J. Aycock and N. Horspool, "Simple generation of static single-assignment form," in *Proc. Int. Conf. Compiler Construction*, Berlin, Heidelberg:Springer, 2000, pp. 110–125.

[27] J. de Fine Licht, A. Kuster, T. De Matteis, T. Ben-Nun, D. Hofer, and T. Hoefler, "StencilFlow: Mapping large stencil programs to distributed spatial computing systems," in *Proc. 2021 IEEE/ACM Int. Symp. Code Gener. Optim.*, 2021, pp. 315–326.

[28] J. Kieffer and G. Ashiotis, "PyFAI: A python library for high performance azimuthal integration on GPU," in *Proc. 7th Eur. Conf. Python Sci.*, 2014, pp. 3–10.

[29] L. Barba and G. Forsyth, "CFD python: The 12 steps to navier-stokes equations," *J. Open Source Educ.*, vol. 2, no. 16, pp. 21, 2019, doi: 10.21105/jose.00021.

[30] P. Mocz, "nbody-python: Create your own n-body simulation (with python)," 2020. [Online]. Available: https://github.com/pmocz/nbody-python

[31] C. Stieger, A. Szabo, T. Bunjaku, and M. Luisier, "Ab-initio quantum transport simulation of self-heating in single-layer 2-D materials," *J. Appl. Phys.*, vol. 122, no. 4, 2017, Art. no. 045708, doi: 10.1063/1.4990384.

[32] S. Behnel et al., "Cython for numpy users," 2025. [Online]. Available: https://cython.readthedocs.io/en/latest/src/userguide/numpy_tutorial.html

[33] Anaconda Inc., "A 5 minute guide to numba," 2012. [Online]. Available: https://numba.readthedocs.io/en/stable/user/5minguide.html

[34] N. P. Rougier, "rougier/from-python-to-numpy: Version 1.1. zenodo," Dec. 2016, doi: 10.5281/zenodo.225783. [Online]. Available: https://zenodo.org/records/225783

[35] Øystein Sture, "Implementation of crc16 (crc-16-ccitt) in python," 2018. [Online]. Available: https://gist.github.com/oysstu/68072c44c02879a2abf94ef350d1c7c6

[36] Anaconda Inc., "Example: Histogram," 2017. [Online]. Available: https://numba.pydata.org/numba-examples/examples/density_estimation/histogram/results.html

[37] G. Bengtsson, "Development of stockham fast fourier transform using data-centric parallel programming," Ph.D. dissertation, 2020. [Online]. Available: http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-287731

[38] Y. LeCun et al., "Backpropagation applied to handwritten zip code recognition," *Neural Comput.*, vol. 1, no. 4, pp. 541–551, 1989, doi: 10.1162/neco.1989.1.4.541.

[39] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. 2016 IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[40] M. Baldauf, A. Seifert, J. Förstner, D. Majewski, and M. Raschendorfer, "Operational convective-scale numerical weather prediction with the COSMO model: Description and sensitivities," *Monthly Weather Rev.*, 139, pp. 3387–3905, 2011.

[41] COSMO, "Consortium for small-scale modeling," Oct. 1998. [Online]. Available: http://www.cosmo-model.org

[42] L.-N. Pouchet et al., "PolyBench: The polyhedral benchmark suite," 2012. [Online]. Available: http://www.cs.ucla.edu/pouchet/software/polybench

[43] P. S. Foundation, "Pep 465–A dedicated infix operator for matrix multiplication," 2014. [Online]. Available: https://www.python.org/dev/peps/pep-0465/

[44] S. Guelton, "Pythran," 2012. [Online]. Available: https://github.com/serge-sans-paille/pythran

[45] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, New York, NY, USA, 2015, Art. no. 73, doi: 10.1145/2807591.2807644.

[46] B. Efron, "Bootstrap methods: Another look at the jackknife," in *Breakthroughs in Statistics*, Berlin, Germany: Springer, 1992, pp. 569–593.

[47] Preferred networks, inc. and preferred infrastructure, inc, "Cupy: User-defined kernels," 2015. [Online]. Available: https://docs.cupy.dev/en/stable/user_guide/kernel.html

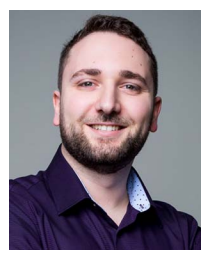
[48] J. de Fine Licht, M. Besta, S. Meierhans, and T. Hoefler, "Transformations of high-level synthesis codes for high-performance computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 5, pp. 1014–1029, May 2021.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

[49] A. Ivanov, N. Dryden, T. Ben-Nun, S. Li, and T. Hoefler, "Data movement is all you need: A case study on optimizing transformers," in *Proc. 4th Conf. Mach. Learn. Syst.*, 2021, pp. 711–732.
[50] NetLib, "Pblas," 1995. [Online]. Available: http://www.netlib.org/scalapack/pblas_qref.html
[51] T. Schneider, R. Gerstenberger, and T. Hoefler, "Application-oriented ping-pong benchmarking: How to assess the real communication overheads," *Computing*, vol. 96, no. 4, pp. 279–292, Apr. 2014, doi: 10.1007/s00607-013-0330-4.
[52] NetLib, "Blacs," 1995. [Online]. Available: https://www.netlib.org/blacs/
[53] Nvidia, "Legate: High productivity performance computing," 2021. [Online]. Available: https://github.com/nv-legate
[54] Anaconda Inc., "Dask array," 2014. [Online]. Available: https://docs.dask.org/en/latest/array.html
[55] G. Kwasniewski, M. Kabić, M. Besta, J. VandeVondele, R. Solcà, and T. Hoefler, "Red-blue pebbling revisited: Near optimal parallel matrix-matrix multiplication," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2019, pp. 1–22.
[56] M. Bauer, S. Treichler, E. Slaughter, and A. Aiken, "Legion: Expressing locality and independence with logical regions," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2012, pp. 1–11.
[57] D. Bonachea and P. H. Hargrove, "GASNet-EX: A high-performance, portable communication library for exascale," in *Proc. Lang. Compilers Parallel Comput.*, Springer International Publishing, 2018, pp. 138–158. [Online]. Available: https://www.springer.com/us/book/9783030346263
[58] P. S. Foundation, "Pep 318–decorators for functions and methods," 2003. [Online]. Available: https://www.python.org/dev/peps/pep-0318/
[59] C. Osuna, T. Wicky, F. Thuering, T. Hoefler, and O. Fuhrer, "Dawn: A high-level domain-specific language compiler toolchain for weather and climate applications," *Supercomput. Front. Innovations*, vol. 7, no. 2, pp. 79–97, 2020. [Online]. Available: https://www.superfri.org/superfri/article/view/314
[60] E. Slaughter, W. Lee, S. Treichler, M. Bauer, and A. Aiken, "Regent: A high-productivity programming language for HPC with logical regions," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal.*, New York, NY, USA, 2015, Art. no. 81, doi: 10.1145/2807591.2807629.
[61] J. Hegarty et al., "Darkroom: Compiling high-level image processing code into hardware pipelines," *ACM Trans. Graph.*, vol. 33, no. 4, pp. 144:1–144:11, Jul. 2014.
[62] V. Clement et al., "The CLAW DSL: Abstractions for performance portable weather and climate models," in *Proc. Platform Adv. Sci. Comput. Conf.*, New York, NY, USA, 2018, Art. no. 2, doi: 10.1145/3218176.3218226.
[63] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis transformation," in *Proc. Int. Symp. Code Gener. Optim.*, 2004, pp. 75–86.
[64] C. Lattner et al., "MLIR: Scaling compiler infrastructure for domain specific computation," in *Proc. 2021 IEEE/ACM Int. Symp. Code Gener. Optim.*, 2021, pp. 2–14.
[65] T. Gysi et al., "Domain-specific multi-level IR rewriting for GPU," 2020, *arXiv: 2005.13014*.
[66] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," in *Proc. 34th ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, New York, NY, USA, 2013, pp. 519–530, doi: 10.1145/2491956.2462176.
[67] US Department of Energy, "Definition - performance portability," 2016. [Online]. Available: https://performanceportability.org/perfport/definition/
[68] L. Dagum and R. Menon, "OpenMP: An industry-standard API for shared-memory programming," *IEEE Comput. Sci. Eng.*, vol. 5, no. 1, pp. 46–55, First Quarter 1998, doi: 10.1109/99.660313.
[69] O. Organization, "OpenACC," 2021. [Online]. Available: https://www.openacc.org/
[70] K. Group, "OpenCL," 2021. [Online]. Available: https://www.khronos.org/opencl
[71] Khronos Group, "Sycl," 2021. [Online]. Available: https://www.khronos.org/sycl
[72] B. Ashbaugh et al., "Data parallel C: Enhancing SYCL through extensions for productivity and performance," in *Proc. Int. Workshop OpenCL*, New York, NY, USA, 2020, Art. no. 7, doi: 10.1145/3388333.3388653.
[73] B. Chamberlain, D. Callahan, and H. Zima, "Parallel programmability and the chapel language," *Int. J. High Perform. Comput. Appl.*, vol. 21, no. 3, pp. 291–312, Aug. 2007, doi: 10.1177/1094342007078442.
[74] C. Rice University, "High performance fortran language specification," *SIGPLAN Fortran Forum*, vol. 12, no. 4, pp. 1–86, Dec. 1993, doi: 10.1145/174223.158909.
[75] H. C. Edwards, C. R. Trott, and D. Sunderland, "Kokkos: Enabling manycore performance portability through polymorphic memory access patterns," *J. Parallel Distrib. Comput.*, vol. 74, no. 12, pp. 3202–3216, 2014.
[76] LLNL, "RAJA performance portability layer," 2019. [Online]. Available: https://github.com/LLNL/RAJA
[77] S. Guelton, "Numpy benchmarks," 2019. [Online]. Available: https://github.com/serge-sans-paille/numpy-benchmarks
[78] A. N. Ziogas, T. Ben-Nun, T. Schneider, and T. Hoefler, "NPBench: A benchmarking suite for high-performance numPy," in *Proc. ACM Int. Conf. Supercomput.*, New York, NY, USA, 2021, pp. 63–74, doi: 10.1145/3447818.3460360.
[79] F. Broquedis et al., "hwloc: A generic framework for managing hardware affinities in HPC applications," in *Proc. 18th Euromicro Conf. Parallel Distrib. Netw.-Based Process.*, 2010, pp. 180–186.

Alexandros Nikolaos Ziogas is a postdoctoral researcher with ETH Zurich. His research interests lie in algorithms, programming models, performance modeling, and performance optimization for high-performance computing and how those apply to relevant scientific applications, such as computational nanoelectronics. He received the ACM Gordon Bell Prize 2019.

Timo Schneider received the Diploma degree from the Chemnitz University of Technology. He is a staff scientist with ETH Zurich. His main research topics are programming models and communication within high-performance computing, particularly hardware offload for collective communication.



Tal Ben-Nun (Member, IEEE) is a staff scientist with Lawrence Livermore National Laboratory. His research interests involve the intersections between high-performance computing, machine learning, and programming languages. Current research projects include data-centric parallel programming models, large-scale distributed machine learning for scientific computing applications, and advancing machine comprehension of code via deep learning.

Alexandru Calotoiu received the PhD degree from Technische Universitat (TU) Darmstadt, in 2017, and was awarded the prize for best PhD thesis of the year in the Computer Science Department for outstanding scientific performance by the Association of Friends of Technische Universitat zu Darmstadt e.V. He works as a postdoctoral fellow with Prof. Hoefler's group with ETH Zurich. His research interests are data-centric optimization, performance modeling and engineering, parallel programming, and machine learning. He is the main designer of the automated performance-modeling tool Extra-P.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.

Tiziano De Matteis is an assistant professor with the Computer Science Department of Vrije Universiteit Amsterdam. His Principal research interests are related to Parallel and Distributed Computing with a focus on systems and programming models for spatial and reconfigurable computing for HPC, post-Moore architectures, and sustainability in modern computing systems.

Johannes de Fine Licht received the PhD degree from the Scalable Parallel Computing Laboratory, ETH Zurich under the supervision of Prof. Torsten Hoefler, where he conducted research on productively programming FPGAs for HPC. He is a compiler engineer with NextSilicon, where he works on a novel MLIR-based compiler targeting NextSilicon's adaptive dataflow chip for accelerating HPC applications. Throughout his PhD, he ran a successful tutorial series on FPGA programming with high-level synthesis along with Prof. Hoefler that was presented at SC18 – SC21, contributing to bridge the gap between hardware development and the HPC community. His work included collaborations on FPGA programming with Microsoft and AMD/Xilinx research divisions. He contributes to MLIR/LLVM and maintains several popular open-source repositories. His interests are in novel spatial computing architectures, programming models, and compilation systems for HPC.

Luca Lavarini received the both bachelor's and master's degrees in computational science from ETH Zurich, where he specialized in high-performance computing and artificial intelligence. He contributed to several research projects and won the ETH Willi-Studer award when graduating, in 2021. Currently, he is working as a software engineer, focusing on the design and optimization of large-scale distributed systems in data-intensive environments.

Torsten Hoefler (Fellow, IEEE) is a professor of computer science with ETH Zurich, a member of Academia Europaea, and a fellow of the ACM, and ELLIS. His research interests revolve around the central topic of "Performance-centric System Design" and include scalable networks, parallel programming techniques, and performance modeling. He won best paper awards with the ACM/IEEE Supercomputing Conference SC10, SC13, SC14, SC19, SC22, SC23, HPDC'15, HPDC'16, IPDPS'15, and other conferences. He published hundreds of peer-reviewed scientific conference and journal articles and authored chapters of the MPI-2.2 and MPI-3.0 standards. He received the IEEE CS Sidney Fernbach Award, the ACM Gordon Bell Prize, the ISC Jack Dongarra award, the Latsis prize of ETH Zurich, as well as the German Max-Planck Humboldt Medal. Additional information about his can be found on his homepage at htor.ethz.ch.

Authorized licensed use limited to: ULAKBIM UASL - IZMIR YUKSEK TEKNOLOJI ENSTITUSU. Downloaded on February 08,2026 at 00:30:48 UTC from IEEE Xplore. Restrictions apply.